

Liste chaînée en C

```
typedef struct Maillon {  
    unsigned long valeur;  
    struct Maillon *next;  
} Maillon;
```

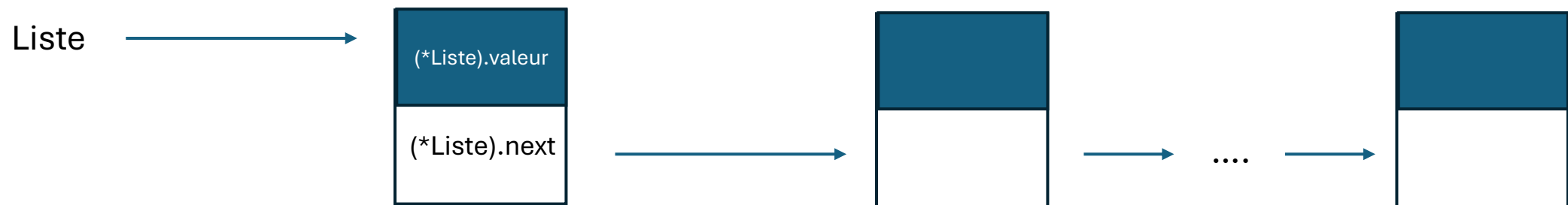
```
Maillon *Liste;
```

Liste est un pointeur vers un objet de type Maillon

*Liste est l'objet de type Maillon

(*Liste).valeur est le contenu de type unsigned long du champ valeur (synonyme Liste->valeur)

(*Liste).next est un pointeur vers un objet de type Maillon



```
Maillon *Liste;
```

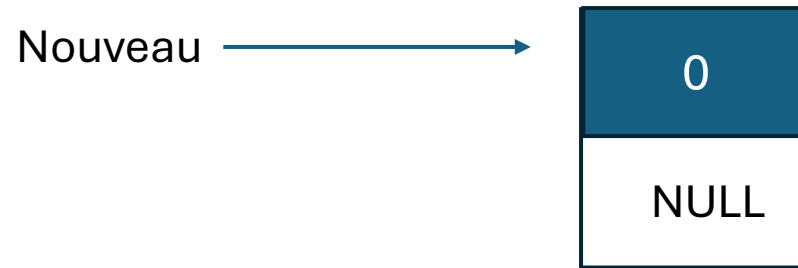
```
Liste = NULL;
```

La liste est vide , elle ne contient aucun élément

Insertion simple d'un nouvel élément en début de liste

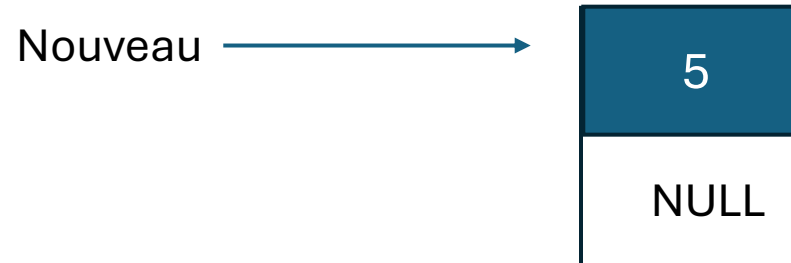
```
Maillon *Nouveau;
```

```
Nouveau = calloc(1,sizeof(Maillon)); //allocation en mémoire d'une zone pour stocker un Maillon
```

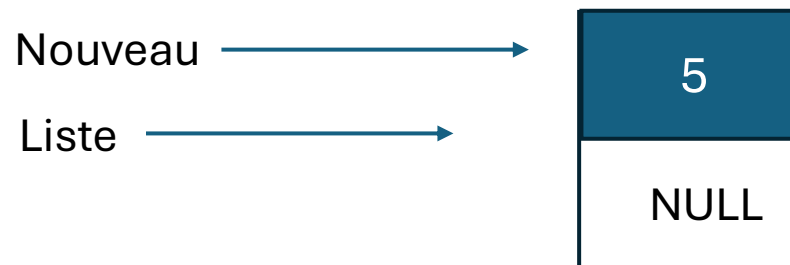


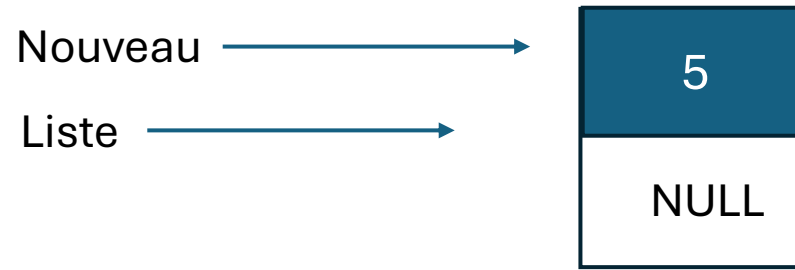
```
(*Nouveau).valeur = 5;
```

```
(*Nouveau).next = Liste;
```



```
Liste = Nouveau;
```



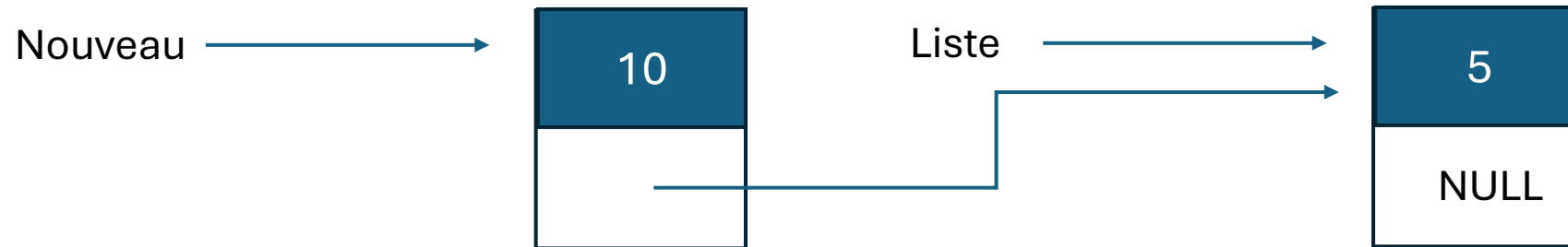


Insertion d'un 2^{ème} élément

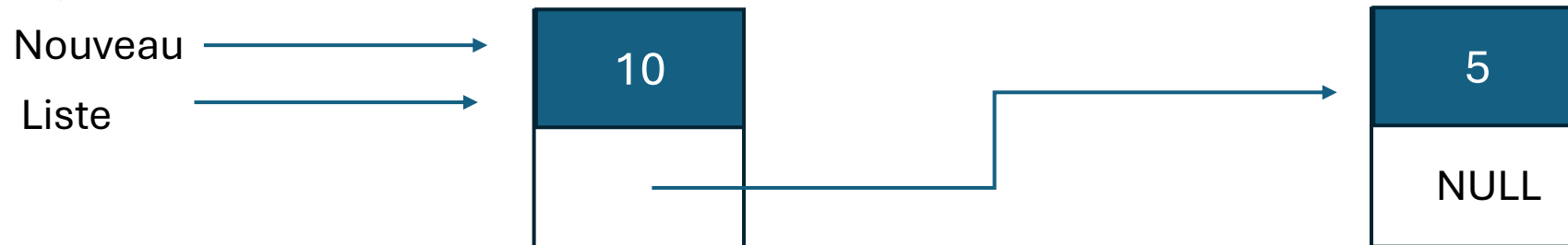
`Nouveau = calloc(1, sizeof(Maillon)); //allocation en mémoire d'une zone pour stocker un Maillon`



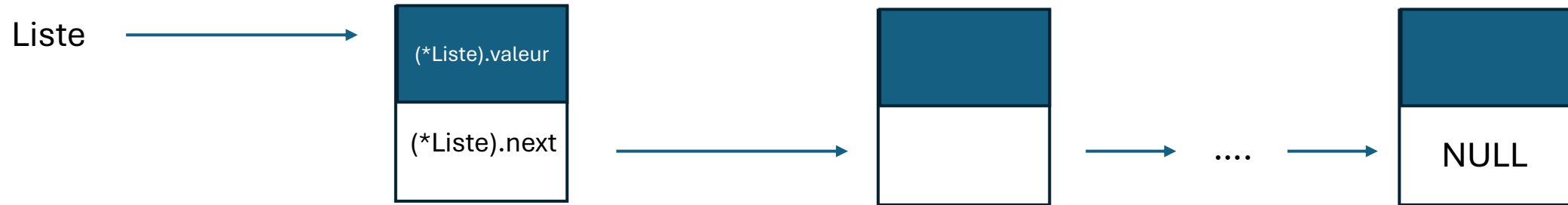
`(*Nouveau).valeur = 10;`
`(*Nouveau).next = Liste;`



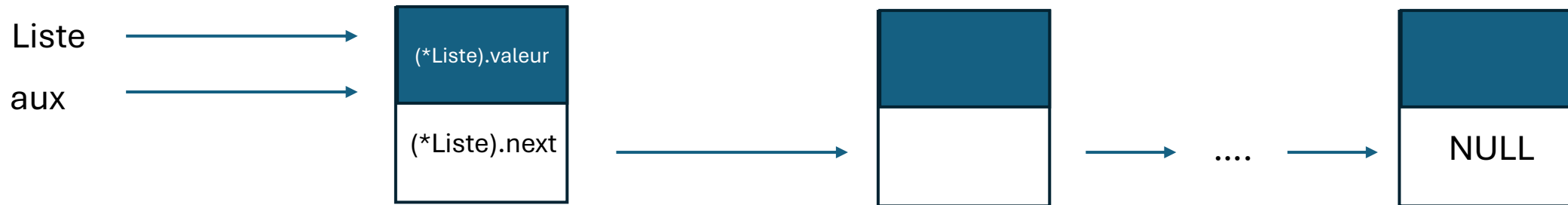
`Liste = Nouveau;`



Parcours de la liste



Maillon *aux;
aux = Liste;



```
while (aux != NULL) {  
    printf("%d\n", aux->valeur);  
    aux = aux->next;  
}
```

Recherche d'un élément de valeur v dans la liste

```
Maillon *aux;  
int oncontinue;  
aux = Liste;
```

```
oncontinue = 1;  
while ((aux!= NULL) && (oncontinue)){  
    oncontinue = (aux->valeur != v);  
    aux = aux->next;  
}  
if (oncontinue != 1)  
    printf(« valeur présente dans la liste »);  
else  
    printf(« valeur non présente dans la liste »);
```

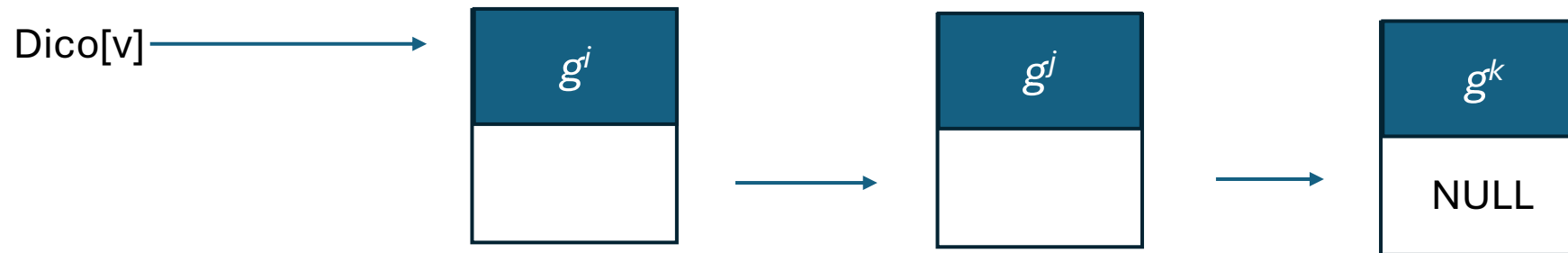
Liste chaînée en C

```
typedef struct Maillon {  
    unsigned long valeur;  
    struct Maillon *next;  
} Maillon;  
  
Maillon *Liste;
```

Dictionnaire en C

Objectif : associer à une valeur v , une liste chaînée

Application au log discret : si $H(g^i)=H(g^j)=H(g^k)=v$, on aura dico[v] qui sera un pointeur vers la liste chaînée composée des maillons contenant g^i, g^j, g^k .



Dico est un tableau dynamique de listes chaînées qui contiendra au plus $\lfloor \sqrt{p} \rfloor$ entrées

```
typedef struct Maillon {
```

```
unsigned long gj;
```

```
Unsigned long j;
```

```
struct Maillon *next;
```

```
} Maillon;
```

```
Maillon **Dico ;
```

```
Dico = calloc ( $\lfloor \sqrt{p} \rfloor$ , sizeof(Maillon *)); //Allocation du dictionnaire , tous les Dico[i] valent NULL
```

Au cours du temps

